Day 6 - Errors, Modules, npm and Packages

Day 6 — Errors, Modules, npm and Packages

Objectives

- Handle errors with `try`/`catch`/`finally`, and create your own custom error types
- Understand JavaScript's modern module system (`import`/`export`) and how it differs from the older CommonJS (`require`) style you'll also encounter
- Understand npm and `package.json` — how JavaScript projects declare and install their dependencies

`try` / `catch` / `finally`

```
try {
    const result = riskyOperation();
    console.log(result);
} catch (error) {
    console.log("Something went wrong:", error.message);
} finally {
    console.log("This always runs, error or not.");
}
```

`try` runs the code that might fail; the instant something inside it throws an error, execution jumps immediately to `catch`, skipping any remaining lines in the `try` block. `error` (you can name this variable anything, but `error` or `err` is conventional) is the actual error object, and `error.message` gives you its human-readable description. `finally` always runs afterward, whether or not an error happened — useful for cleanup code that must run either way.

Throwing your own errors

```
function withdraw(balance, amount) {
    if (amount > balance) {
        throw new Error(`Insufficient funds: have ${balance}, need ${amount}`);
    }
    return balance - amount;
}

try {
    withdraw(100, 500);
} catch (error) {
    console.log(error.message);   // "Insufficient funds: have 100, need 500"
}
```

`throw new Error("message")` creates a new error object and immediately raises it — if nothing catches it, Node prints it and stops the program, exactly like Python's uncaught exceptions.

Custom error types

For errors specific to your own program's logic, define your own error class (full class syntax tomorrow, Day 8 — this specific pattern is simple enough to use today) by **extending** JavaScript's built-in `Error`:

```
class InsufficientFundsError extends Error {
    constructor(message) {
        super(message);            // pass the message up to the built-in Error's own setup
        this.name = "InsufficientFundsError";   // gives it a distinct name, shown in stack traces
    }
}

function withdraw(balance, amount) {
    if (amount > balance) {
        throw new InsufficientFundsError(`need ${amount}, have ${balance}`);
    }
    return balance - amount;
}

try {
    withdraw(100, 500);
} catch (error) {
    if (error instanceof InsufficientFundsError) {
        console.log("Please add funds to your account.");
    } else {
        throw error;   // re-throw anything we didn't specifically expect -- don't silently swallow
    }
}
```

`instanceof` checks whether an object was created from a particular class (or one of its subclasses) — you'll get the full treatment of `class`, `extends`, and `instanceof` tomorrow and the day after; for today, just recognize this pattern: defining your own error type lets calling code distinguish "the specific error I expected" from "something else entirely went wrong," and handle each appropriately, rather than blindly catching and hiding everything.

Never write an empty `catch` block that does nothing — this is JavaScript's equivalent of Python's dangerous bare `except: pass`, and it's just as dangerous here: it silently swallows every possible error, including ones you never anticipated, leaving you with no clue later about what actually went wrong.

Modules — splitting code across multiple files

A **module** is any single JavaScript file, treated as a self-contained unit whose specific pieces (functions, variables, classes) can be shared with, or "exported" to, other files. Modern JavaScript uses **ES modules** (the `import`/`export` syntax) — this is what you should default to writing:

```
// mathUtils.js
export function add(a, b) {
    return a + b;
}

export function subtract(a, b) {
    return a - b;
}

export const PI = 3.14159;

// main.js
```

```
import { add, subtract, PI } from "./mathUtils.js";

console.log(add(2, 3));   // 5
```

`export` in front of a function, variable, or class marks it as available to other files; anything without `export` stays private to that one file. `import { add, subtract } from "./mathUtils.js"` pulls in exactly the named things you ask for, from the given file path (the `./` means "in this same folder").

You can also have one **default export** per file — useful when a module's whole purpose is to provide one main thing:

```
// logger.js
export default function log(message) {
    console.log(`[LOG] ${message}`);
}

// main.js
import log from "./logger.js";   // no curly braces for a DEFAULT import -- and you can name it anyt
log("Hello");   // "[LOG] Hello"
```

CommonJS — the older module style you'll still see constantly

Plain Node.js scripts (like every `exercises.js`/`solutions.js` file you've written this week) traditionally use a different, older system called **CommonJS**, using `require` and `module.exports` instead of `import`/`export`:

```
// mathUtils.js (CommonJS style)
function add(a, b) {
    return a + b;
}
module.exports = { add };

// main.js (CommonJS style)
const { add } = require("./mathUtils.js");
console.log(add(2, 3));   // 5
```

This is exactly the pattern you've already been using all week, at the bottom of every `solutions.js` file (`module.exports = { ... }`) — now you know what it means and why it's there. **You should know both systems, because both are extremely common in real, existing codebases** (CommonJS in older Node projects and much existing tooling; ES modules in modern projects and virtually all browser-based code) — but for brand-new code, ES modules (`import`/`export`) is the modern standard, and what you'll use starting with TypeScript on Day 11.

npm and `package.json` — installing and managing other people's code

npm ("Node Package Manager") is how you install code that someone else has already written and published, so you don't have to write everything yourself from scratch. Every JavaScript/Node project has a `package.json` file at its root, describing the project and exactly which external packages it depends on:

```
npm init -y            # creates a starter package.json for a new project (the -y accepts all the de
npm install lodash        # downloads the "lodash" package and adds it as a dependency in package.js
```

This creates (or updates) a file that looks roughly like:

```
{
```

```
  "name": "my-project",
  "version": "1.0.0",
  "dependencies": {
    "lodash": "^4.17.21"
  }
}
```

It also creates a `node_modules/` folder, where the actual downloaded package code lives, and a `package-lock.json` file, which records the *exact* versions of everything installed (including packages your direct dependencies themselves depend on), so the install can be reproduced identically on another machine. `node_modules/` **should never be committed to git** — it's large, entirely derived from `package.json`, and gets recreated automatically by running `npm install` on any machine that already has the `package.json` and `package-lock.json` files (you'll set up a proper `.gitignore` for this on Day 13).

Once installed, you use a package exactly like any of your own modules:

```
const _ = require("lodash");        // CommonJS style
// or: import _ from "lodash";        // ES module style, if your project is configured for it

console.log(_.capitalize("hello"));   // "Hello"
```

You'll install and use your first real packages for real starting Day 11 (a testing tool, and eventually the TypeScript compiler itself).

Exercises

Open `exercises.js`, fill in each `// TODO`, and run `node exercises.js`.